Research Paper Analysis and Recommendation System

Helps researchers find what to read next on arXiv, with answers they can check

Group C

Ching-Yun Pan [48841331]

Teepakorn Wongwatanachai [48382493]

Guillermo Fernandez Gonzalez [48936596]

My Di Luong [48982830]

COMP8460 Assignment 3: Use Case 3 · Week 13, November 2026

Problem and goal

There are too many new papers on arXiv to read them all. A student starting on a new topic can search by keyword and read dozens of abstracts to find the few that matter, or ask an LLM, which is faster but unreliable: in our test only **40%** of its sentences were supported by the papers.

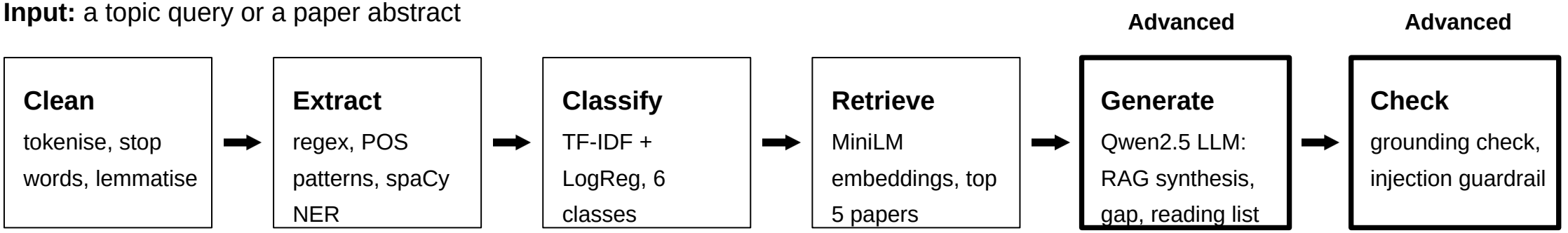
Given a topic or a paper abstract, our system tells the user:

1. which research area the paper belongs to
2. its key phrases, methods and datasets
3. five related papers to read next
4. a short summary
5. a possible research gap, with the sources cited

The LLM answers only from the five papers it retrieves and must cite them by number, so the user can check each claim.

System pipeline

Input: a topic query or a paper abstract



Notebook 02
My Di Luong

Notebook 02
My Di Luong

Notebook 03
Guillermo Fernandez Gonzalez

Notebook 04
Ching-Yun Pan

Notebook 05
Teepakorn Wongwatanachai

Notebooks 05, 06
Teepakorn Wongwatanachai

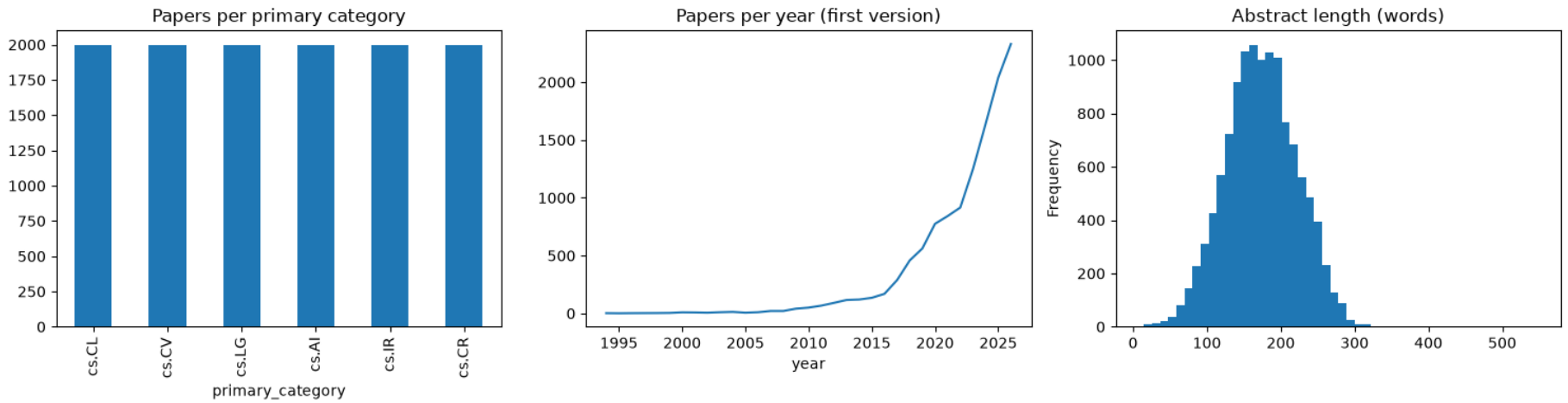
Output: domain, key phrases, 5 related papers, summary, research gap, share of grounded claims

Notebook 01: the shared data sample and split (My Di Luong)

Notebook 06: joins the stages into one system (Teepakorn Wongwatanachai)

Dataset

- arXiv metadata snapshot from Kaggle
- **12,000** CS papers, **2,000** from each of 6 computer science categories: cs.AI, cs.CL, cs.CR, cs.CV, cs.IR, cs.LG
- **80/20** train/test split, seed 42
- Every later notebook reads only this sample



Sample overview (notebook 01)

Notebook 02 · My Di Luong

Preprocessing

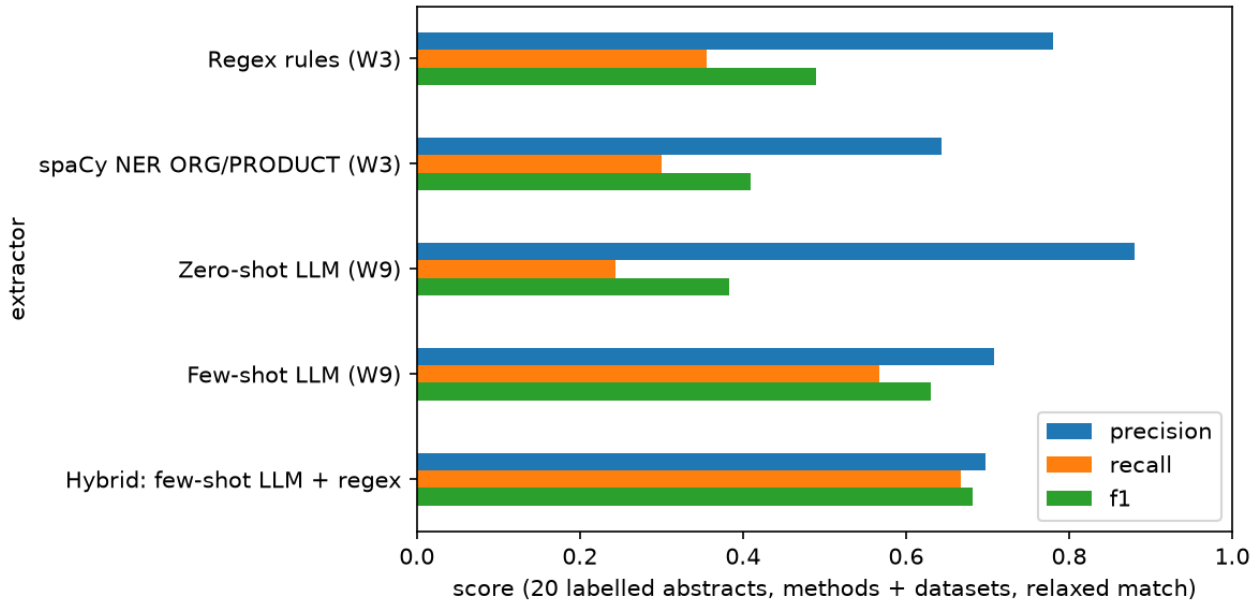
- **Clean the text:** lowercase, split into words, remove stop words like “the” and “and”, then lemmatise
- Lemmatisation keeps real words: **8,466** words left on 1,000 abstracts
- Stemming makes it smaller (6,483) but creates non-words like “analys”, **so I keep lemmas**

Step	**Vocabulary**
Raw lowercase words	11,417
Cleaned: no stop words, punctuation or numbers	10,818
Cleaned + Porter stemming	**6,483**
Cleaned + spaCy lemmatisation	**8,466**

Vocabulary size on 1,000 abstracts (notebook 02)

Notebook 02 · My Di Luong

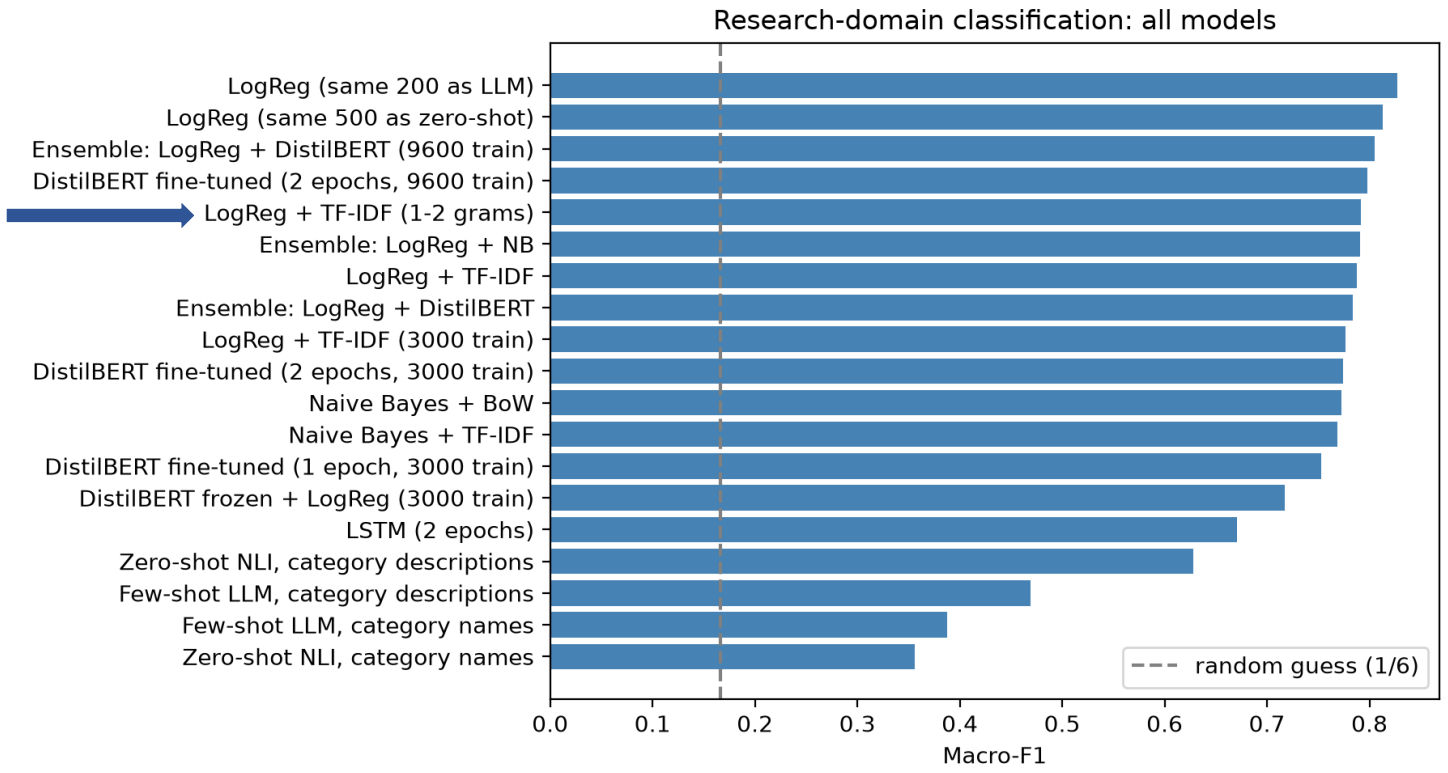
Information extraction results



Precision, recall and F1 on **20 abstracts** (notebook 02)

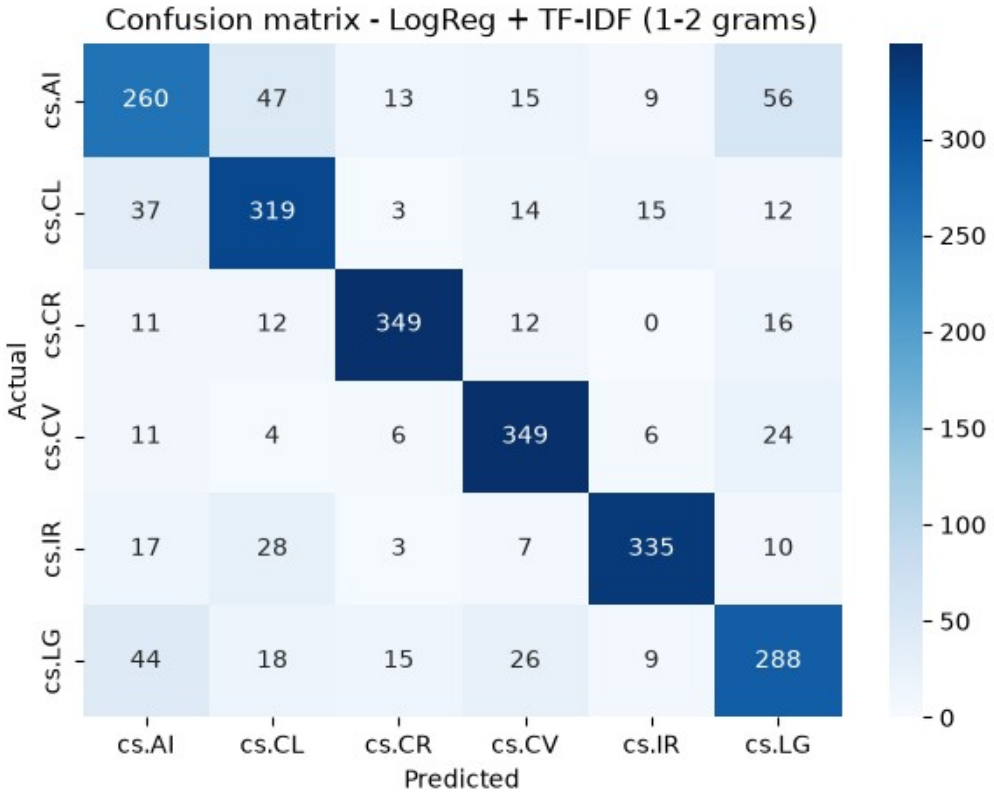
- **Test set:** labelled methods and datasets in 20 abstracts (90 names); AI drafted the labels and checked
- **Rules:** regex is precise but misses lowercase names (F1 0.489); spaCy NER is not trained on scientific terms (0.409)
- Zero-shot LLM is the most precise (0.88) but finds only a quarter of the names
- Three examples raise recall to 0.57 and F1 to 0.63
- **Best score:** LLM + regex, F1 0.682, because they miss different things
- **Limitation:** 20 abstracts is a small test set, so these numbers have wide uncertainty

Domain classification results



- **DistilBERT** ties our pick on all **9,600** papers (**0.798**, up from **0.753**); averaging the two best tries reaches **0.805**.
- We keep **LogReg + TF-IDF (1-2 grams)**, macro-F1 **0.792**: fast, no GPU, explainable.
- **Zero-shot NLI** went from **0.356** to **0.628** by just giving short category descriptors.
- **Few-shot LLM** was slow, inaccurate and required a GPU, macro-F1 **0.469.**

Where the classifier goes wrong



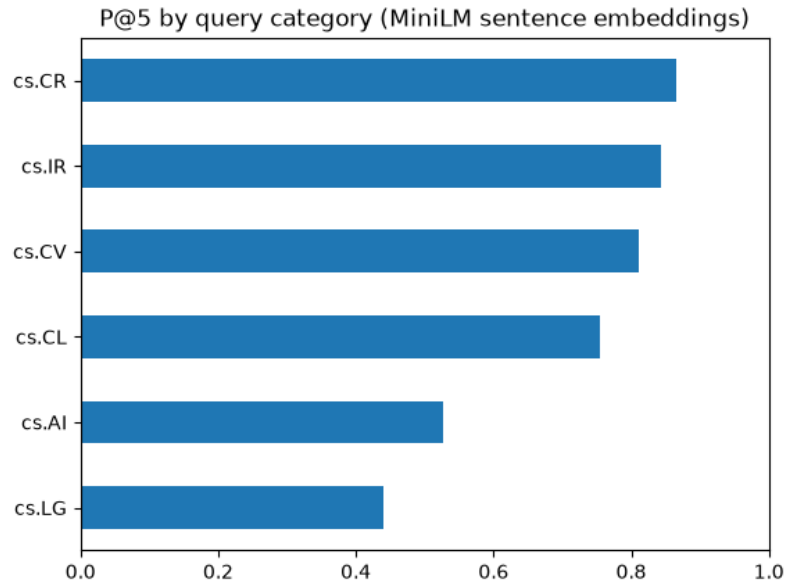
- Most errors are between cs.AI, cs.LG and cs.CL (confusion matrix, left).
- Cross-listed papers are harder: **0.763** accuracy against **0.828** for single-category papers.
- On text as short as a title, macro-F1 falls to **0.70**, so the label on a typed query is a rough guide.

Notebook 04 · Ching-Yun Pan

Paper recommendation (retrieval)

Retriever	P@5
MiniLM embeddings	**0.704**
TF-IDF (1-2 grams)	**0.619**
Averaged word vectors	**0.420**
Random baseline	**0.167**

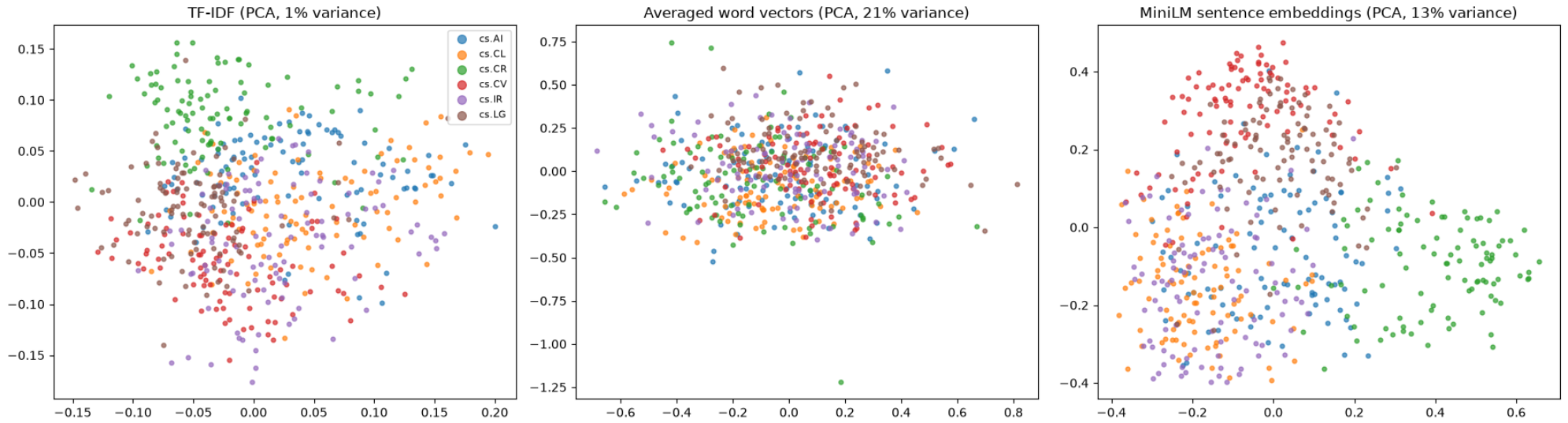
- Relevant = same primary category as the query paper
- MiniLM is best and takes under **0.1 seconds** per query
- cs.LG is hardest (P@5 **0.44**): ML overlaps with every other area



MiniLM P@5 by category (notebook 04)

Notebook 04 · Ching-Yun Pan

Topic discovery with clustering



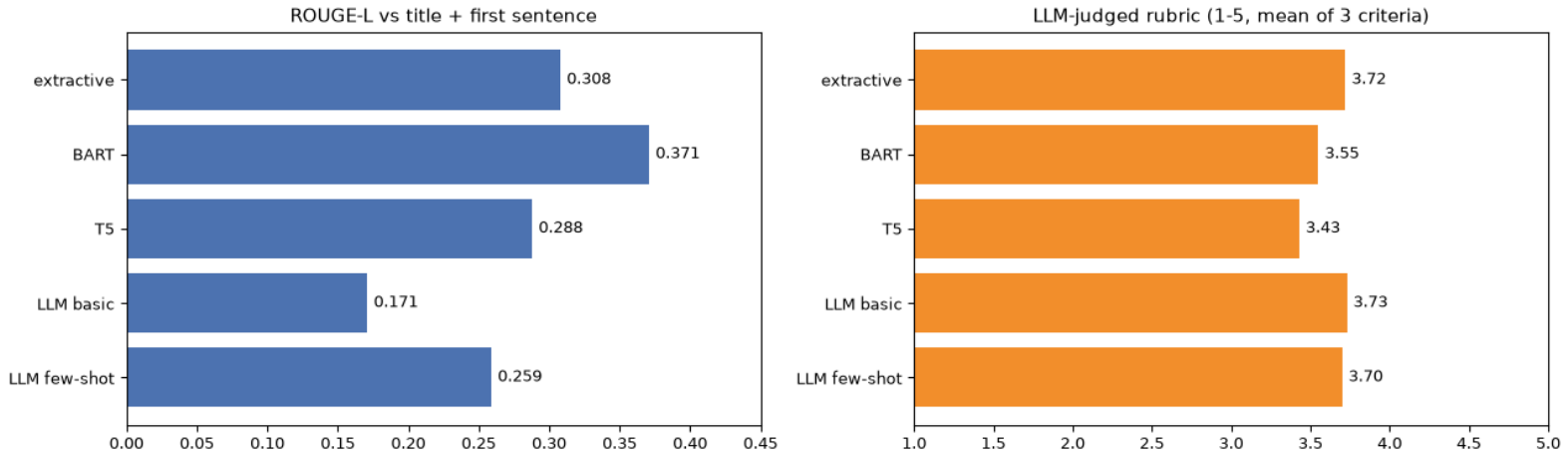
600 training papers projected to 2D with PCA, coloured by category (notebook 04)

KMeans, k = 6, all 9,600 training papers	TF-IDF	Avg. word vectors	MiniLM
Purity	0.572	0.296	**0.727**
Adjusted Rand index	0.263	0.046	**0.484**

MiniLM cluster top terms:

- cs.IR (92%) recommendation, user, item
- cs.CR (89%) security, attacks, privacy
- cs.CV (84%) image, 3D, video
- cs.CL (56%, least pure) language, models, retrieval, LLMs

Summarisation results



Left: ROUGE-L. Right: LLM-judged rubric, 1 to 5. Same 20 test papers (notebook 05)

- ROUGE-L uses the title + first sentence as reference, so it rewards copying: BART is highest (**0.371**)
- The LLM judge cannot separate the top three: basic LLM (**3.73**), extractive (**3.72**), few-shot LLM (**3.70**)
- Few-shot examples cut LLM summaries from **118** to **51** words, with none cut off

```
# notebook 05, cell [13]: few-shot summary prompt (2 worked examples)
def prompt_few_shot(abstract):
    shots = "".join(f"Abstract: {a}\nSummary: {s}\n\n" for a, s in FEW_SHOT_EXAMPLES)
    return ("Summarize each abstract in two sentences: problem + method, then the main result. "
            f"Use only facts from the abstract.\n\n{shots}Abstract: {abstract}\nSummary:")
```

Notebook 05 · Teepakorn Wongwatanachai

Research gaps (CoT) and RAG synthesis

Chain-of-thought gap analysis

Same 10 papers, three prompts for the research gap:

Prompt	Reasoned	Specificity	Judge 1–5
Direct (no CoT)	n/a	**0.37**	**3.3**
Zero-shot CoT	0/10	0.38	3.3
Few-shot CoT	10/10	0.17	3.0

Specificity = share of the gap's words found in the abstract. CoT did not help the 1.5B model, so the system uses the **direct prompt**.

```
# notebook 05, cell [17]: the direct gap prompt we kept (no CoT)
def prompt_gap_direct(abstract):
    return ("Read the abstract and state the most important "
            "research gap it leaves open. "
            "Write 'GAP:' followed by the gap in one or two "
            f"sentences.\n\nAbstract: {abstract}")
```

RAG synthesis

Example question: “How can large language models be defended against prompt injection attacks?”

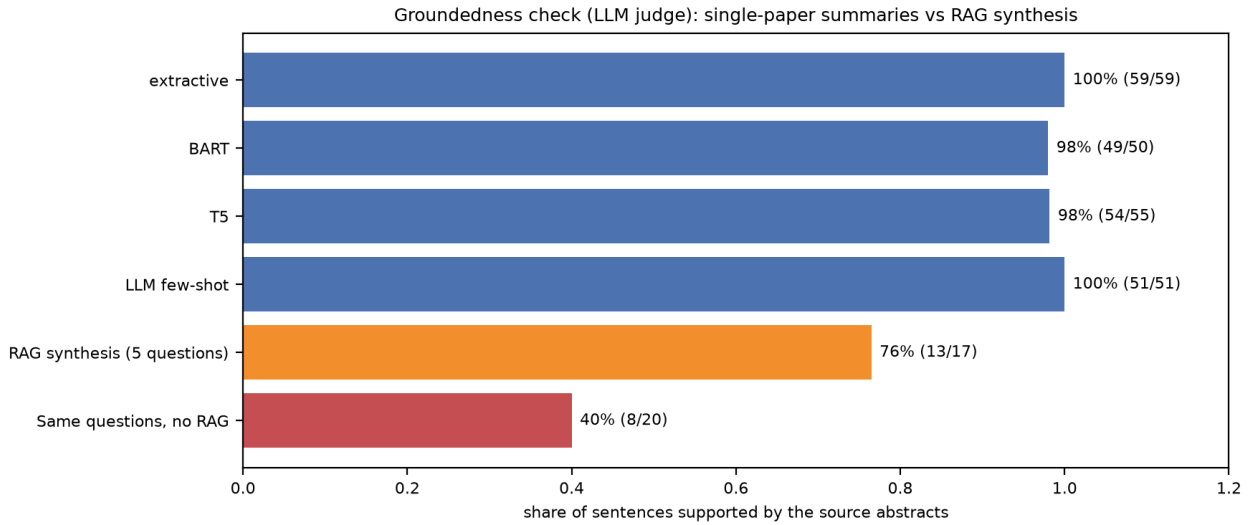
1. MiniLM retrieves the top 5 abstracts
2. The abstracts go into the prompt, numbered [1] to [5]
3. The LLM writes a synthesis with citations, one gap and reading suggestions
4. Each claim is checked against the abstracts

5 questions	Supported	Cite a paper
No retrieval	8/20 (40%)	0/20
RAG	13/17 (76%)	5/17
RAG, cite every sentence	**18/21 (86%)**	**10/21**

Synthesis sentences judged against the retrieved abstracts (notebooks 05, 05.1)

Notebooks 05 and 06 · Teepakorn Wongwatanachai

Hallucination and prompt injection



Sentences supported by the source abstracts, judged by the LLM (notebook 05)

Prompt injection test

- We hid 3 different instructions in 3 abstracts: 9 attempts per setting

Defence	Known attacks	New attacks
None	2/9	4/9
Guardrail prompt	3/9	4/9
Filter + guardrail	**0/9**	**4/9**

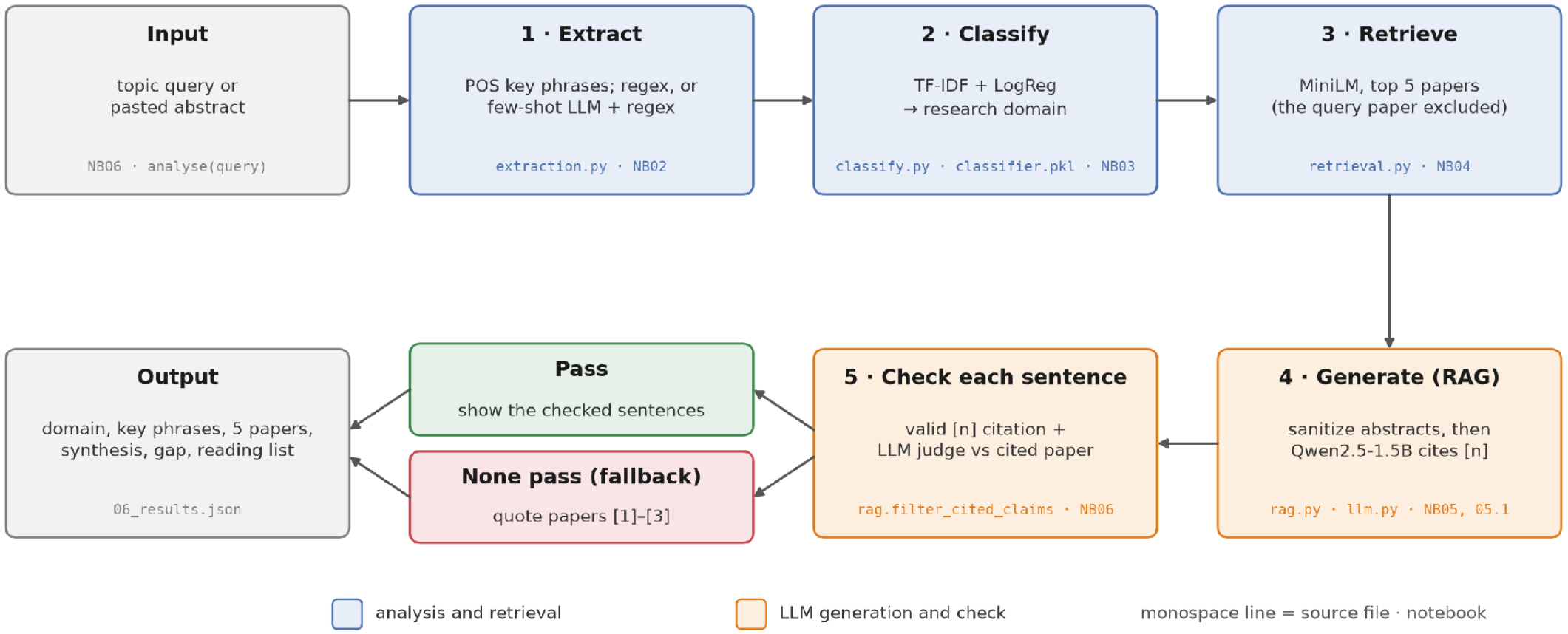
Attacks that worked. The filter only stops attacks like the ones it was built from: injection stays an **open risk**.

Single-paper summaries are almost fully grounded; RAG synthesis reaches **76%**, and only **40%** without retrieval.

```python
# rag.py: the LLM judge checks one sentence at a time against its source
JUDGE_SYSTEM = "You are a strict fact checker. Answer with exactly one word: SUPPORTED or UNSUPPORTED."

def is_supported(claim, source):
    reply = llm.generate(f"Source text:\n{source}\n\nClaim: {claim}\n\nIs every fact in the claim stated in the source text?",
                         system=JUDGE_SYSTEM, max_new_tokens=5)
    return re.fullmatch(r"SUPPORTED[.!]?", reply.strip(), flags=re.IGNORECASE) is not None
```

Demo workflow: one function, every notebook



Notebook 06 · Teepakorn Wongwatanachai

Demo: one query through the system

Step	Output (notebook 06)
Query	“defending large language models against prompt injection and jailbreak attacks”
Key phrases	large language models, prompt injection, jailbreak attacks
Domain	cs.CR (cryptography and security)
Retrieved	5 papers: **4 in cs.CR**, 1 in cs.LG
Synthesis	None of the LLM's 3 sentences passed the citation check, so the system shows 3 sentences quoted from papers [1] to [3], plus one research gap and reading suggestions
Grounded	Generated: **1 of 3** sentences supported. Shown: **3 of 3** exact quotes. Steps before the LLM: **2.5 s**
Larger model	Qwen2.5-3B: cites **26 of 29** sentences across 8 questions, no fallback needed

06_end_to_end_demo.ipynb · cell [8] · analyse("defending large language models against prompt injection and jailbreak attacks")

```
QUERY: defending large language models against prompt injection and jailbreak attacks
domain: cs.CR   key phrases: ['large language models', 'prompt injection', 'jailbreak attacks']
    methods/datasets: []

RETRIEVED:
  [1] 0.78 [cs.CR] 2409.00137 Emerging Vulnerabilities in Frontier Models: Multi-Turn Jailbreak Attacks
  [2] 0.77 [cs.CR] 2408.04811 h4rm3l: A language for Composable Jailbreak Attack Synthesis
  [3] 0.74 [cs.LG] 2505.11790 JULI: Jailbreak Large Language Models by Self-Introspection
  [4] 0.74 [cs.CR] 2407.01902 SeqAR: Jailbreak LLMs with Sequential Auto-Generated Characters
  [5] 0.73 [cs.CR] 2506.10597 SoK: Evaluating Jailbreak Guardrails for Large Language Models

EXTRACTIVE EVIDENCE FALLBACK: [1] Large language models (LLMs) are improving at an exceptional rate. [2] Despite
    their demonstrated valuable capabilities, state-of-the-art (SOTA) widely deployed large language models (LLMs)
    still have the potential to cause harm to society due to the ineffectiveness of their safety filters, which can
    be bypassed by prompt transformations called jailbreak attacks. [3] Large Language Models (LLMs) are trained
    with safety alignment to prevent generating malicious content.
  dropped (no valid citation): All five papers collectively highlight the importance of studying both
      single-turn and multi-turn jailbreak at
  dropped (no valid citation): They emphasize the need for more comprehensive datasets and frameworks to
      address the weaknesses in current sa
  dropped (not supported by cited paper): Papers like [1], [2], and [3] specifically focus on introducing
      new techniques and datasets to improve LLM def

GAP (not independently checked): None of the papers directly address the issue of prompt injection attacks,
    which involve injecting specific inputs to manipulate the output of an LLM. This type of attack is distinct from
    traditional jailbreak attacks and requires a separate set of defenses and analyses.

RECOMMENDATIONS (reasons not independently checked):
[1] "Emerging Vulnerabilities in Frontier Models: Multi-Turn Jailbreak Attacks" - Provides a detailed
    dataset and methodology for evaluating jailbreak attacks in both single and multi-turn formats, highlighting
    the importance of considering structural differences in security.

[2] "h4rm3l: A language for Composable Jailbreak Attack Synthesis" - Introduces h4rm3l, a DSL for composing
    jailbreak attacks, addressing the lack of a formal representation for jailbreak attacks and enabling
    efficient synthesis of complex attacks.

[3] "JULI: Jailbreak Large Language Models by Self-Introspection" - Proposes a method to jailbreak LLMs
    using token log probabilities, demonstrating superior effectiveness compared to existing SOTA approaches and
    providing insights into the feasibility of self-introspection-based defenses.

synthesis grounded 100% of 3 claims; citation coverage 100%; exact-source citations 3/3; verified=True (exact
    source match); format_ok=True
pipeline before LLM 2.47 s; all LLM work 0.0 s (RAG response cached: True)
No generated synthesis sentence passed the per-claim check; gap and recommendation reasons remain unverified.
```

Limitations and future work

Limitations

- “Relevant” is approximated by same category
- Only 20 abstracts, with AI-drafted labels, for extraction
- RAG synthesis: **76%** of sentences supported (40% without retrieval)
- No citation graph in the arXiv metadata
- cs.AI, cs.LG and cs.CL overlap a lot

Future work

- Human relevance labels for test queries
- Citations from the Semantic Scholar API
- A larger LLM, or reranking before synthesis
- Hybrid retrieval: TF-IDF + MiniLM
- Multi-label domain classification

Techniques used, by notebook

Notebook	Owner	Basic (Weeks 1 to 6)	Advanced (Weeks 7 to 12)
01 Data	My Di Luong	Shared 12,000-paper sample and 80/20 split	
02 Extraction	My Di Luong	Preprocessing; POS tagging, spaCy NER and regex for key phrases	Prompt engineering and few-shot prompting
03 Classification	Guillermo Fernandez Gonzalez	TF-IDF + Logistic Regression	Fine-tuning DistilBERT; ensemble classifiers
04 Retrieval	Ching-Yun Pan	Sentence embeddings + cosine similarity; KMeans clustering	Retriever for RAG
05 LLM	Teepakorn Wongwatanachai	Extractive summary (TF-IDF sentence scoring)	Foundation LLM, run locally + few-shot and chain-of-thought prompting + RAG; groundedness check and injection guardrail
06 Demo	Teepakorn Wongwatanachai	Joins the stages into one system	

Comparison and final choices

Component	Options compared	Final choice	Reason
Extraction	regex, spaCy NER, zero-shot and few-shot LLM	Few-shot LLM + regex	best F1 (0.682)
Classification	NB, LogReg, LSTM, DistilBERT, NLI, LLM, ensembles	LogReg + TF-IDF	macro-F1 0.792, ties DistilBERT; fast, no GPU
Retrieval	TF-IDF, averaged word vectors, MiniLM	MiniLM	best P@5 (0.704), under 0.1 seconds
Summary	extractive, BART, T5, basic and few-shot LLM	Few-shot LLM	most readable, fully grounded
Research gap	zero-shot and few-shot CoT, RAG synthesis	CoT + RAG	cites sources, claims can be checked

Overall: simple methods performed best for classification and retrieval // LLMs were the best for extraction and free-text output.

For the user: one query gives the field, the key methods, five papers to read next, and a summary and gap that cite those papers, instead of a keyword list to read through.

Thank you

Questions?

Ching-Yun Pan [48841331],
Teepakorn Wongwatanachai [48382493],
Guillermo Fernandez Gonzalez [48936596]
My Di Luong [48982830]